



SISTEMA DE GESTÃO DE FALHAS INDUSTRIAIS

DISCIPLINA: ENGENHARIA DE SOFTWARE

Documento Arquitetural

Refatoração com Padrões de Projeto (GoF)

Autores:

Anna Lívia Vasconcelhos, Gabriel Dias,
Lucas Sanches Marcilio, Mirella Costa
Viana e Thalyson Gama da Costa

Funções:

Refatoração Arquitetural

v1.0.4 - 02 de junho de 2026

Resumo

Este documento descreve a refatoração do diagrama de classes do Sistema de Gestão de Falhas de Manutenção Industrial mediante a aplicação de cinco padrões de projeto do catálogo *Gang of Four* (GoF). Para cada padrão são apresentados: o problema identificado no projeto original, a solução proposta, a justificativa técnica, o impacto arquitetural, as vantagens e limitações, e o trecho atualizado do diagrama de classes em imagens. Os padrões escolhidos — **State**, **Strategy**, **Observer**, **Factory Method** e **Facade** — cobrem as três categorias do catálogo (comportamental, criacional e estrutural). Por restrição do trabalho, não foram utilizados os padrões *Abstract Factory*, *Adapter* e *Decorator*.

Índice

1	Visão Geral da Refatoração	4
2	Evolução do Diagrama de Classes	5
2.1	Versão Original (Parte 1, corrigida)	5
2.2	Versão Evoluída (com Padrões de Projeto)	5
2.3	Principais Alterações Estruturais	5
3	Padrão State (Comportamental)	6
3.1	Problema identificado	6
3.2	Solução proposta	6
3.3	Justificativa técnica e por que usar	6
3.4	Impacto arquitetural	6
3.5	Vantagens e limitações	6
3.6	Trecho do diagrama de classes	7
4	Padrão Strategy (Comportamental)	8
4.1	Problema identificado	8
4.2	Solução proposta	8
4.3	Justificativa técnica e por que usar	8
4.4	Impacto arquitetural	8
4.5	Vantagens e limitações	8
4.6	Trecho do diagrama de classes	8
5	Padrão Observer (Comportamental)	9
5.1	Problema identificado	9
5.2	Solução proposta	9
5.3	Justificativa técnica e por que usar	9
5.4	Impacto arquitetural	9
5.5	Vantagens e limitações	9
5.6	Trecho do diagrama de classes	9
6	Padrão Factory Method (Criacional)	10
6.1	Problema identificado	10
6.2	Solução proposta	10
6.3	Justificativa técnica e por que usar	10
6.4	Impacto arquitetural	10
6.5	Vantagens e limitações	10
6.6	Trecho do diagrama de classes	10
7	Padrão Facade (Estrutural)	11
7.1	Problema identificado	11
7.2	Solução proposta	11
7.3	Justificativa técnica e por que usar	11
7.4	Impacto arquitetural	11
7.5	Vantagens e limitações	11
7.6	Trecho do diagrama de classes	12
8	Conclusão	13

1 Visão Geral da Refatoração

O diagrama original concentrava grande quantidade de responsabilidades na classe **Falha** e utilizava enumerações (**StatusChamada**, **Prioridade**) carregadas de regras de negócio implícitas. A criação dos diversos subtipos de **Funcionario** ocorria de forma direta, e a orquestração do fluxo de atendimento ficava a cargo do cliente. Esses pontos representavam *code smells* clássicos: classe inchada, condicionais espalhados, alto acoplamento e baixa coesão.

A Tabela 1 resume os cinco padrões aplicados e o *smell* que cada um resolve.

Padrão	Categoria	Problema que resolve
State	Comportamental	Transições de statusChamada controladas por condicionais espalhados.
Strategy	Comportamental	Regra fixa e rígida de priorização dentro de Falha .
Observer	Comportamental	Acoplamento direto entre Falha e os serviços de alerta/notificação.
Factory Method	Criacional	Instanciação direta (new) dos subtipos de Funcionario .
Facade	Estrutural	Cliente obrigado a orquestrar múltiplos objetos do subsistema.

Tabela 1: Padrões aplicados e respectivos problemas.

Por que estes cinco padrões?

A escolha priorizou padrões que eliminam dores reais já presentes no diagrama, e não padrões “encaixados à força”. Cada padrão remove um acoplamento ou uma rigidez específica, e o conjunto cobre as três categorias do catálogo GoF, demonstrando domínio das diferentes intenções de projeto.

2 Evolução do Diagrama de Classes

2.1 Versão Original (Parte 1, corrigida)

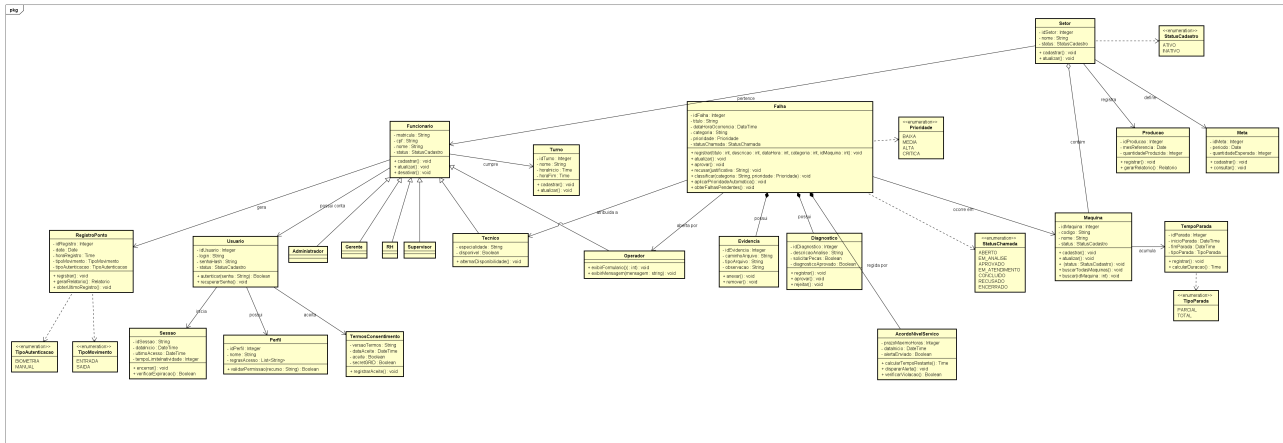


Figura 1: Diagrama de classes original do sistema, antes da aplicação dos padrões de projeto.

2.2 Versão Evoluída (com Padrões de Projeto)

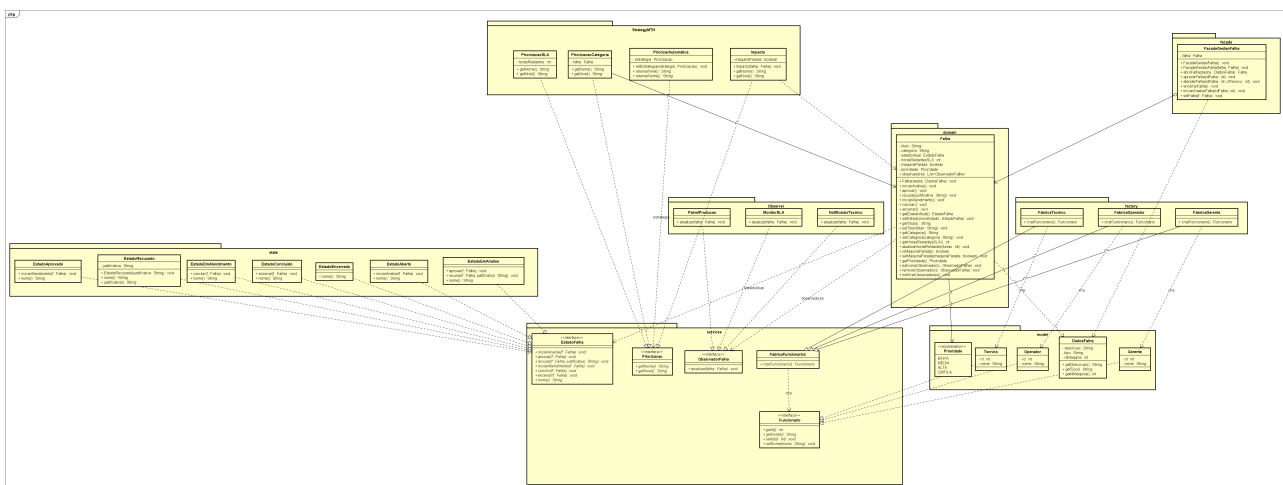


Figura 2: Diagrama de classes evoluído, reorganizado em pacotes e com os cinco padrões aplicados.

2.3 Principais Alterações Estruturais

- **Organização em pacotes:** o modelo, antes monolítico, foi modularizado nos pacotes **factory**, **pessoas**, **observer**, **facade**, **domain**, **model**, **state** e **StrategyMTH**, separando responsabilidades por padrão.
- **State:** o ciclo de vida da Falha, antes representado apenas pelo enum **StatusChamada**, foi substituído pela interface **EstadoFalha** e sete estados concretos (**EstadoAberto** a **EstadoEncerrado**), eliminando condicionais de transição.
- **Strategy:** a priorização, antes embutida no método **classificar()** da **Falha**, foi extraída para a interface **Priorizacao** com três políticas intercambiáveis (**PriorizacaoCategoria**, **Impacto** e **PriorizacaoSLA**), gerenciadas pela classe contexto **PriorizarAutomatica**.
- **Observer:** a **Falha** tornou-se um sujeito observável, notificando **MonitorSLA**, **NotificadorTecnico** e **PainelProducao** via interface **ObservadorFalha**, removendo o acoplamento direto a esses módulos.

- **Factory Method:** a criação de funcionários foi centralizada na hierarquia `FabricaFuncionarios` (com `FabricaOperador`, `FabricaTecnico` e `FabricaGerente`), padronizando a instanciação dos subtipos via o método `criarFuncionario()`.
- **Facade:** foi introduzida a `FacadeGestaoFalha` como ponto de entrada único para o fluxo de gestão de falhas, encapsulando as interações entre domínio, estados e notificações.
- **Enxugamento do modelo:** classes periféricas da Parte 1 sem papel no fluxo de falhas (`RegistroPonto`, `Sessao`, `Perfil`, `Turno`, `Producao`, `Meta` etc.) foram removidas do recorte para destacar o núcleo afetado pelos padrões.

3 Padrão State (Comportamental)

3.1 Problema identificado

A classe `Falha` possuía o atributo `statusChamada` do tipo enumeração. Toda regra de transição — por exemplo, “só é possível aprovar uma falha que esteja em análise” — era implementada com estruturas condicionais (`if/switch`) espalhadas pelos métodos `aprovar()`, `recusar()` e `concluir()`. Isso tornava o código frágil, propenso a estados inválidos e difícil de estender.

3.2 Solução proposta

Cada valor do enumerado foi transformado em uma classe de estado (`EstadoAberto`, `EstadoEmAnalise`, `EstadoAprovado`, etc.) que implementa a interface `EstadoFalha`. Uma classe abstrata `EstadoBase` fornece implementação padrão para todos os métodos da interface (lançando `IllegalStateException`), e cada estado concreto sobrescreve apenas as transições válidas a partir de si mesmo. A `Falha` passa a **delegar** a operação ao estado atual.

3.3 Justificativa técnica e por que usar

O padrão State é a resposta canônica para objetos cujo comportamento depende de seu estado interno. Ao distribuir a lógica de transição entre classes de estado, elimina-se o condicional central e torna-se **impossível**, por construção, executar uma transição inválida. A adição de um novo estado (por exemplo, `AguardandoPeca`) não exige alterar os estados existentes, respeitando o *Open/Closed Principle*.

3.4 Impacto arquitetural

A responsabilidade de controle de fluxo sai da `Falha` e passa para a hierarquia de estados, aumentando a coesão da entidade. Surge uma nova dimensão de classes, mas cada uma é pequena e de responsabilidade única.

3.5 Vantagens e limitações

- **Vantagens para nosso contexto:** ajuda a eliminar condicionais; transições inválidas viram erro explícito; fácil extensão; cada estado pode/deve ser testável isoladamente.
- **Limitações:** aumenta o número de classes; para máquinas de estado muito simples pode ser excesso de estrutura (*over-engineering*).

3.6 Trecho do diagrama de classes

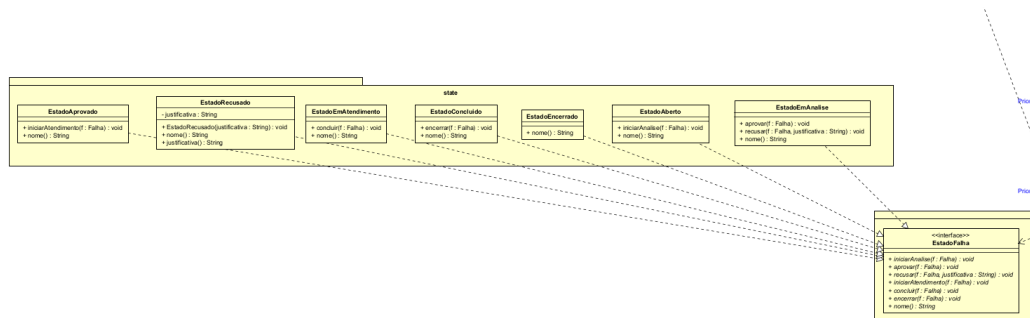


Figura 3: Diagrama de Classes - Padrão State

4 Padrão Strategy (Comportamental)

4.1 Problema identificado

O método `aplicarPrioridadeAutomatica()` embutia uma única regra fixa de priorização. Contudo, a priorização de falhas industriais varia conforme o contexto: por impacto na produção, por proximidade de violação do SLA ou por categoria. Alterar a política exigia reescrever o corpo do método dentro da *Falha*.

4.2 Solução proposta

O algoritmo de priorização foi extraído para a interface *Priorizacao*, com três implementações intercambiáveis: *PriorizacaoCategoria*, *Impacto* e *PriorizacaoSLA*. A classe contexto *PriorizarAutomatica* recebe uma estratégia e delega o cálculo a ela, expondo os métodos `retornarNivel()` e `retornarNome()`.

4.3 Justificativa técnica e por que usar

O padrão Strategy encapsula algoritmos intercambiáveis sob uma interface comum. É a escolha correta quando existem várias formas de executar uma mesma operação e deseja-se trocá-las sem modificar o cliente. Respeita o *Open/Closed Principle* e favorece a testabilidade, pois a estratégia pode ser substituída em tempo de execução.

4.4 Impacto arquitetural

A *Falha* deixa de conhecer detalhes do cálculo de prioridade e passa a depender apenas da abstração. As regras de negócio de priorização ficam isoladas em classes próprias, melhorando a manutenibilidade.

4.5 Vantagens e limitações

- **Vantagens:** troca de algoritmo sem alterar a *Falha*; fácil adição de novas políticas; cada estratégia é testável de forma isolada.
- **Limitações:** introduz mais classes; o cliente precisa conhecer as estratégias disponíveis para escolher a adequada.

4.6 Trecho do diagrama de classes

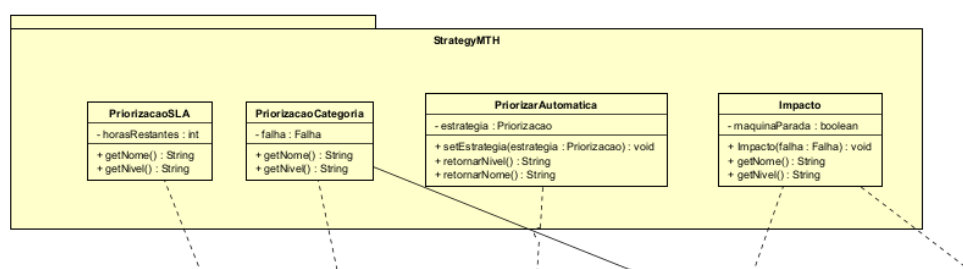


Figura 4: Diagrama de Classes - Padrão Strategy

5 Padrão Observer (Comportamental)

5.1 Problema identificado

Quando uma falha mudava de situação, vários componentes precisavam reagir: o monitor de SLA verificava violações e disparava alertas, o técnico era notificado e o painel de produção era atualizado. Se a **Falha** chamasse cada serviço diretamente, ficaria fortemente acoplada a todos eles, e cada novo interessado obrigaria a editar a **Falha**.

5.2 Solução proposta

A **Falha** tornou-se o *Subject*. Os interessados implementam a interface **ObservadorFalha** — **MonitorSLA**, **NotificadorTecnico** e **PainelProducao** — e registram-se na falha. Ao mudar de estado, o método **notificarObservadores()** avisa todos os observadores sem conhecer seus tipos concretos.

5.3 Justificativa técnica e por que usar

O padrão Observer define uma dependência um-para-muitos de modo que, ao mudar o estado de um objeto, todos os dependentes sejam notificados automaticamente. É ideal quando o número de interessados é variável e não deve ser fixado em tempo de compilação, promovendo baixo acoplamento entre emissor e receptores.

5.4 Impacto arquitetural

A **Falha** depende apenas da interface **ObservadorFalha**, não das implementações. Novos observadores podem ser adicionados ou removidos em tempo de execução, sem qualquer alteração na entidade observada.

5.5 Vantagens e limitações

- **Vantagens:** desacoplamento total entre **Falha** e os serviços reativos; registro dinâmico de observadores; aderência ao princípio da inversão de dependência.
- **Limitações:** a ordem de notificação não é garantida; em caso de muitos observadores, pode haver impacto de desempenho ou notificações em cascata difíceis de depurar.

5.6 Trecho do diagrama de classes

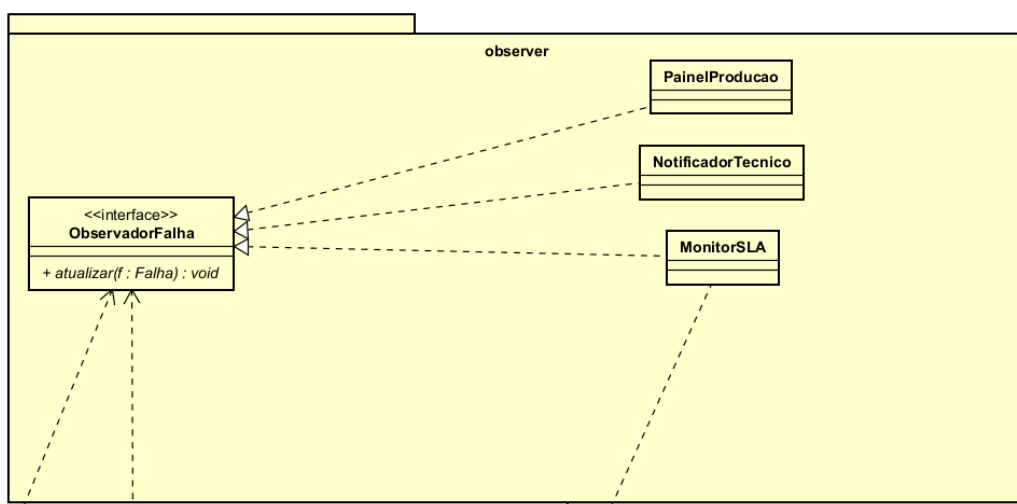


Figura 5: Diagrama de Classes - Padrão Observer

6 Padrão Factory Method (Criacional)

6.1 Problema identificado

O sistema possui diversos subtipos de `Funcionario` (`Tecnico`, `Operador`, `Gerente`, entre outros). A criação direta com `new` espalhava pelo código a lógica comum de inicialização (definir `status = ATIVO`, validar matrícula, atribuir nome), gerando duplicação e dificultando a manutenção.

6.2 Solução proposta

Criou-se a classe abstrata `FabricaFuncionarios` com o método abstrato `criarFuncionario()`, que cada subclasse (`FabricaTecnico`, `FabricaOperador` e `FabricaGerente`) implementa para instanciar o tipo concreto correspondente (`Tecnico`, `Operador` ou `Gerente`), todos implementando a interface `Funcionario`.

6.3 Justificativa técnica e por que usar

O Factory Method delega às subclasses a decisão sobre qual objeto concreto criar, mantendo a lógica de inicialização comum em um único ponto. Optou-se por Factory Method (e não Abstract Factory) porque o objetivo é criar **um produto por vez** de uma única hierarquia (`Funcionario`), e não famílias de produtos relacionados — além de Abstract Factory ser vedado pelo escopo do trabalho.

6.4 Impacto arquitetural

A regra de criação fica centralizada, reduzindo duplicação. A instanciação passa a ocorrer exclusivamente através das fábricas, padronizando o processo e desacoplando o cliente dos tipos concretos.

6.5 Vantagens e limitações

- **Vantagens:** elimina duplicação de inicialização; centraliza a criação; facilita a adição de novos tipos de funcionário.
- **Limitações:** cada novo tipo exige uma nova subclasse de fábrica, aumentando o número de classes do projeto.

6.6 Trecho do diagrama de classes

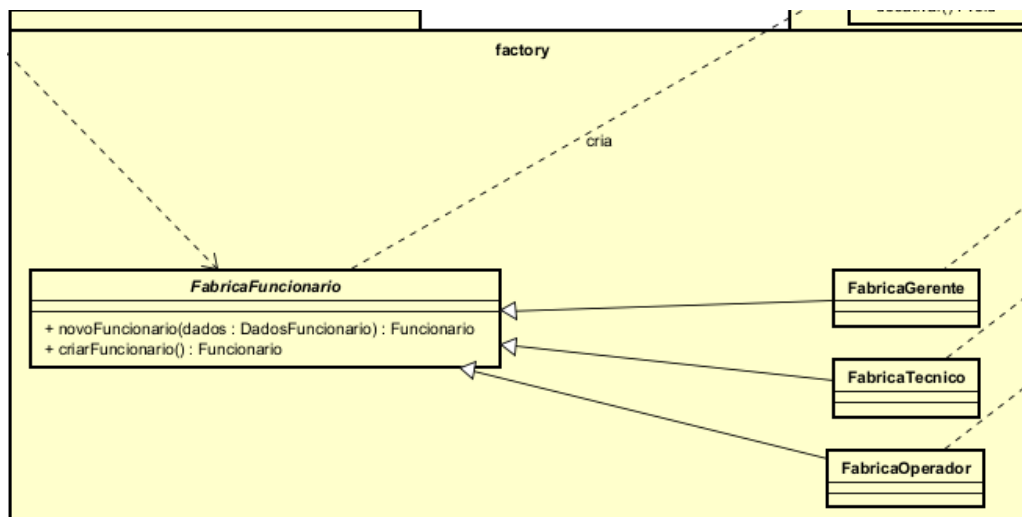


Figura 6: Diagrama de Classes - Padrão Factory Method

7 Padrão Facade (Estrutural)

7.1 Problema identificado

O fluxo de abertura e tramitação de uma falha envolve coordenar vários objetos: criar a **Falha**, aplicar a estratégia de prioridade, registrar observadores e gerenciar as transições de estado. Exigir que o cliente (controlador/interface) orquestrasse tudo isso o acoplava às entranhas do subsistema.

7.2 Solução proposta

Criou-se a *FacadeGestaoFalha*, que expõe operações de alto nível (*abrirFalha*, *aprovarFalha*, *atenderFalha*, *encerrarFalha*, *iniciarAnaliseFalha*) e esconde a orquestração interna entre as diversas classes do domínio. A fachada mantém internamente um mapa de falhas ativas, referenciando-as pelo identificador.

7.3 Justificativa técnica e por que usar

O padrão Facade fornece uma interface unificada e simplificada para um conjunto de interfaces de um subsistema. É adequado quando se deseja reduzir o acoplamento do cliente com a complexidade interna. A fachada é também o ponto natural de integração dos demais padrões: ela cria a **Falha**, escolhe a *Strategy*, registra os *Observers* e utiliza a *Factory*.

7.4 Impacto arquitetural

O cliente passa a depender de uma única classe de entrada, e não de toda a teia de objetos do domínio. Isso melhora a legibilidade, facilita testes de fluxo e isola futuras mudanças internas do subsistema.

7.5 Vantagens e limitações

- **Vantagens:** reduz o acoplamento do cliente; centraliza a orquestração; oferece um ponto único e estável de acesso.
- **Limitações:** pode evoluir para uma classe “deus” se acumular responsabilidades em excesso; adiciona uma camada de indireção.

7.6 Trecho do diagrama de classes

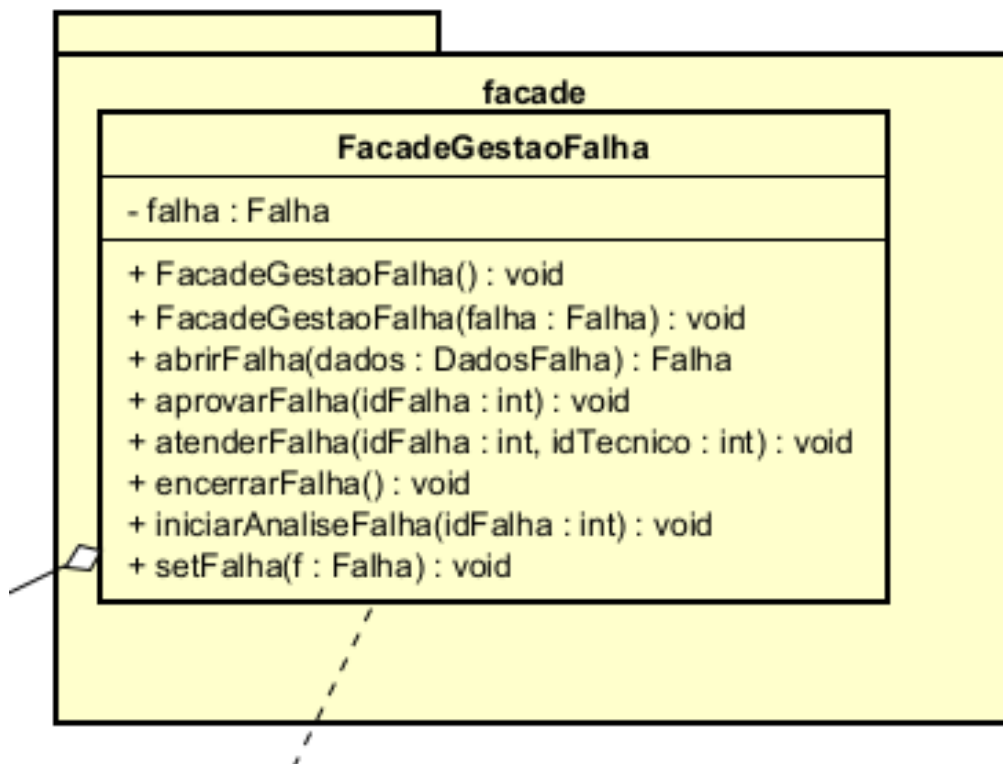


Figura 7: Diagrama de Classes - Padrão Facade

8 Conclusão

A aplicação dos cinco padrões transformou um diagrama com forte configuração de responsabilidades em uma arquitetura coesa e de baixo acoplamento. Os padrões **State** e **Strategy** retiraram a lógica de controle e de cálculo de dentro da **Falha**; o **Observer** eliminou suas dependências de saída; o **Factory Method** centralizou a criação de objetos; e o **Facade** forneceu uma porta de entrada única para o subsistema. O conjunto respeita princípios de projeto consagrados — notadamente o *Single Responsibility Principle* e o *Open/Closed Principle* — resultando em um sistema mais extensível, testável e manutenível, sem recorrer aos padrões vedados pelo escopo (*Abstract Factory*, *Adapter* e *Decorator*).